

AI-Generated Manim Animations & Critical Evaluation

Using Google Generative AI (Gemini) API

In this assignment you will use the Google Generative AI (Gemini) API to automatically generate Manim animation code for mathematical concepts, critically evaluate the quality of the generated code, and publish your work to GitHub with proper documentation.

PART 1 · Manim Code Generation & Critical Analysis

Use the Gemini API (via the google-generativeai Python package) to prompt the model to write Manim animation code for the two tasks below. Run the generated code, observe the output, and write a structured critique.

Visualisation of the Pythagorean Theorem

Prompt the Gemini API to generate a complete Manim scene that visually proves the Pythagorean Theorem ($a^2 + b^2 = c^2$). The animation should label the sides, shade the three squares built on each side, and display the algebraic identity on screen.

What to Submit:

- The Python script used to call the Gemini API (your prompt + API call code)
- The raw Manim code returned by the model (saved as pythagoras.py)
- A rendered video or screenshot of the animation output

Critical Analysis: After running the generated code, identify and explain 5 specific shortcomings in the generated Manim scene. For each point, state what the issue is, why it matters visually or mathematically, and how you would fix it. Consider: incorrect geometry, missing labels, poor animation timing, absence of colour logic, layout overflow, hardcoded values, unused imports, missing wait() calls, etc.

Visualisation of Fourier Series Decomposition

Prompt the Gemini API to generate a Manim scene that demonstrates how a square wave is built up by summing sine harmonics — showing at least the first 5 terms of the Fourier series, each wave drawn in a different colour, and the cumulative sum updating step by step.

What to Submit:

- The Python script containing your Gemini API prompt and call
- The raw Manim code returned by the model (saved as fourier_series.py)
- A rendered video or screenshot showing the wave decomposition

Critical Analysis: Identify and explain 5 specific shortcomings in the generated Fourier scene. Each point must name the exact issue, its impact on the viewer's understanding, and a concrete corrective action. Consider: wave amplitude scaling, axis range mismatch, overlapping labels,

missing legend, colour contrast, incorrect frequency ratios, absent title or annotations, animation speed, etc.

PART 2 · GitHub Repository Submission

Publish your entire work to a public GitHub repository following the structure and documentation standards below. Proper version control and documentation form an integral part of this assessment.

Step-by-Step Instructions

1. Create a public repository. Name it manim-genai-assignment. Initialise with a README.md and add a .gitignore for Python.
 2. Do not flatten everything into the root.
- **Write a thorough README:** Must include: project overview, setup instructions (pip install -r requirements.txt), how to run each Manim scene, and brief findings from your critical analysis.
 - **Make meaningful commits:** Use descriptive commit messages, e.g. add: pythagoras manim scene from gemini or docs: add critique for q1. At least 5 commits are expected.
 - **Submit your repository URL:** Google form will be shared.